

嵌入式第四组作业

思考题

1. 前后台系统在轮询系统的基础上做了什么改变？其运行方式是什么？它适用于什么嵌入式应用？

在轮询系统上做的改变：

传统的轮询系统（Polling System / Super-loop）就是一个纯粹的死循环

（`while(1)`），CPU按顺序挨个检查各个外设的状态，如果事件没发生，CPU也得白白浪费时间去“问”，导致响应延迟大，且无法处理突发事件。

前后台系统（Foreground/Background System）在其基础上引入了中断机制（Interrupt）。它将系统划分为两部分：

- 后台（Background）：就是原来的那个死循环（无限循环），负责处理对时间要求不严格的任务（如屏幕刷新、数据计算、状态指示灯闪烁等）。
- 前台（Foreground）：由各种中断服务程序（ISR）组成，专门负责处理异步的、对实时性要求极高的硬件事件（如按键按下、串口收到数据、定时器溢出）。

运行方式：

1. 系统上电初始化后，主程序进入“后台”死循环持续运行。
2. 当外部硬件事件发生时，硬件触发中断信号。
3. CPU立刻暂停当前正在执行的“后台”代码，保存当前上下文，跳转到对应的“前台”中断服务程序中执行。
4. “前台”快速处理完紧急事件（通常只做最少的工作，如将接收到的数据放入缓冲区，或设置一个标志位）。
5. 退出中断，恢复上下文，CPU回到“后台”刚才被暂停的地方继续执行。后台程序根据前台设置的标志位，再进行后续的复杂处理。

适用的嵌入式应用：

前后台系统是中小型裸机嵌入式开发中最主流的架构。它适用于：

- 复杂度适中、不需要多任务调度的应用：如微波炉控制面板、简单的电机控制器、环境数据采集节点、基于单片机的智能小玩具等。在这些场景中，引入复杂的实时操作系统（RTOS）会显得杀鸡用牛刀，而前后台系统足以保证关键动作（前台）的实时响应。

2. 结合代码分析：IRQ中断发生后，ARM9

S3C2440如何保护现场保护？ 需要保持哪些信息？

在 ARM9（基于 ARM920T 内核）中，中断发生时的“现场保护”由硬件自动完成的部分和软件（汇编代码）手动完成的部分共同组成。

需要保持的信息（现场）：

1. 返回地址：中断结束后，CPU需要知道回到哪里继续执行。
2. 处理器状态：中断发生前 CPU 的工作模式、条件标志位（如运算的进位、溢出标志）及中断屏蔽状态（即 CPSR 寄存器的值）。
3. 通用寄存器：中断服务程序中会用到 R0-R12 等通用寄存器，如果不把它们原有的值存起来，中断里的运算就会破坏原来后台程序的数据。

现场保护的过程与代码分析：

当 IRQ 中断发生时，**ARM**硬件底层会自动做以下事情（对程序员透明）：

1. 将当前程序状态寄存器 **CPSR** 拷贝到 IRQ 模式下的保存状态寄存器 **SPSR_irq** 中。
2. 将下一条要执行的指令地址（此时因流水线原因，PC值为当前中断指令地址+8）保存到 IRQ 模式下的链接寄存器 **LR_irq** 中。
3. 强制进入 IRQ 处理器模式，并关闭 IRQ 中断响应（置位 CPSR 中的 I 位）。
4. 强制 PC 指针跳转到 IRQ 的异常向量地址（通常是 **0x00000018**）。

跳转到异常向量表后，就会进入程序员编写的软件中断服务入口，此时需要通过汇编代码进行深度的现场保护：

IRQ_Handler:

@ 1. 修正返回地址

@ 由于 ARM9 的三级流水线机制，发生中断时 PC 已经指向了当前指令的下一条。

@ 硬件保存的 LR_irq = PC，而我们实际需要返回的地址是 PC-4（被中断的下一条指令）。

```
SUB LR, LR, #4
```

@ 2. 保护现场（压栈）

@ STMFD：满递减堆栈写入指令。

@ 将 R0 到 R12 这些通用寄存器，以及修正后的返回地址 LR，压入 IRQ 模式的私有堆栈中。

```
STMFD SP!, {R0-R12, LR}
```

```
@ -----
```

```
----
```

@ 此时现场已经安全，可以调用 C 语言写的中断处理函数了

```
BL C_Interrupt_Service_Routine
```

```
@ -----
```

```
----
```

@ 3. 恢复现场并返回（出栈）

@ LDMFD：满递减堆栈读取指令。

@ 将之前压入堆栈的 R0-R12 恢复。

@ 关键点：将堆栈中保存的 LR 的值直接弹出给 PC（程序计数器），实现跳转返回。

@ 后缀 '^' 的特殊作用：它指示 CPU 在恢复 PC 的同时，自动将 SPS R_irq 的值恢复到 CPSR 中。

```
LDMFD SP!, {R0-R12, PC}^
```

3.加入AI后，嵌入式系统体系架构有哪些改变？

传统嵌入式系统主要负责“感知（采集）”和“执行（控制）”，逻辑是预先写死的（If-Else）。当 AI（尤其是边缘计算和 TinyML）引入后，设备具备了“认知”和“推理”能力，这引发了体系架构的深刻变革：

1. 硬件架构的变革：走向异构计算 (Heterogeneous Computing)

- 专用AI加速器的引入：传统的 MCU/CPU 算力在处理神经网络（如卷积运算）时效率极低。现在的嵌入式 SoC 通常采用 CPU + NPU（神经网络处理单元）或 CPU + DSP / GPU 的异构架构。CPU 负责系统调度和网络通信，NPU 专门负责张量乘加运算的硬件加速。
- 内存层次结构的重塑：神经网络模型需要占用大量的存储空间来存放权重（Weights）和激活值（Activations）。为了打破“内存墙（Memory Wall）”，新架构极大地增加了片内 SRAM 的容量，甚至引入了存算一体（PIM）技术，以减少数据搬运带来的极高功耗。

2. 软件与中间件架构的变革：增加 AI 推理栈

- 在传统的 OS（Linux/RTOS）和应用程序之间，增加了一层 **AI 推理引擎层（Inference Engine）**。

- 引入了针对微控制器高度优化的框架（如 TensorFlow Lite for Microcontrollers、TVM、NCNN）。这就要求嵌入式开发不仅要懂 C/C++，还需要在 PC 端使用 Python 训练模型，通过量化（Quantization，例如将 32位浮点数压缩为 8位整数）、剪枝等手段，将其编译成能塞进 Flash 的轻量级 `.tflite` 模型文件。

3. 系统拓扑架构的变革：云-边-端协同 (Cloud-Edge-Device Collaboration)

- 传统物联网是“端侧采集数据 → 全量上传云端处理 → 云端下发指令”。这种架构延迟高、极度消耗带宽且存在隐私泄露风险。
- 加入 AI 后，架构变成了：
 - **端侧（设备）**：运行轻量级模型，进行实时、低延迟的初步推理（如语音唤醒、简单的目标检测）。
 - **边缘（网关/边缘服务器）**：运行稍大的模型，处理多设备聚合的复杂数据。
 - **云端**：仅接收端侧经过 AI 提取后的特征数据或异常结果，主要负责大数据汇总、复杂策略制定以及“AI模型的重新训练与通过 OTA 下发更新”。

1. 抢占调度和不可抢占调度的区别是什么？RTOS通常采用上述方式的哪一种？其原因是什么

答：(1) 两种调度方式的区别：

- **不可抢占调度（协作式）**：任务一旦获得CPU执行权，就会一直运行，直到该任务执行完毕或主动阻塞让出CPU。操作系统**无权**强行剥夺其CPU使用权。
- **抢占式调度**：操作系统掌握绝对的调度权。当有更高优先级的任务就绪时，操作系统可以随时强行中断（抢占）当前正在运行的低优先级任务，将CPU分配给高优先级任务。

(2) RTOS的采用方式：

RTOS（实时操作系统）绝大多数采用**基于优先级的抢占式调度**。

(3) 采用该方式的原因：

- **保证硬实时响应（最关键原因）**：嵌入式控制系统中常有危及安全的紧急任务，抢占式调度允许高优先级任务一旦触发，就能立刻打断当前任务瞬间获得CPU，确保最快响应。若采用不可抢占调度，高优先级任务必须等待，会导致无法容忍的延迟。
- **保证系统行为的确定性**：抢占式调度使得高优先级任务的最坏响应时间（WCET）是恒定且可计算的（仅取决于硬件上下文切换开销）。

- **防止系统假死：**避免因某一个低优先级任务陷入死循环或长耗时计算，导致整个系统无法响应其他外部事件。

2.什么是并发性？它与并行性的区别是什么？什么是Safety？它与Security的区别是什么？

答：(1) 并发性及其与并行性的区别：

- **并发性 (Concurrency)：**指系统在**宏观时间段内**具备处理多个任务的能力。在单核CPU中，多个任务通过操作系统的高速切换交替执行，侧重于系统结构设计和任务调度策略。
- **与并行性 (Parallelism) 的区别：**并行性是指系统在**同一物理时刻**真正同时执行多个任务（必须依赖多核或多处理器硬件）。**并发是“同一时间段内交错处理多件事”，并行是“同一时刻同时做多件事”。**并行必定并发，但并发不一定并行。

(2) Safety及其与Security的区别：

- **Safety (功能安全/人身安全)：**关注系统自身的意外故障或随机错误（如硬件老化、软件Bug），目标是确保系统在发生故障时，**不会对外部环境或人类生命财产造成物理伤害**（例如：失效安全机制 Fail-Safe）。
- **与Security (信息安全) 的区别：**Security关注的是外部未授权的**恶意攻击**（如黑客入侵、病毒），目标是**保护系统自身的数据和资产不受窃取或破坏**。两者的根本区别在于**威胁来源和保护方向**：Safety防范“无意”的系统故障伤害世界，Security防范“有意”的外部攻击伤害系统。

3.与通用操作系统相比，RTOS具有哪些特点？请重点对其中的一点进行详细叙述。

答：(1) RTOS相较于通用操作系统 (GPOS) 的主要特点：

- **极强的确定性 (Determinism) 与硬实时响应 (Hard Real-time)。**
- 采用严格的、基于优先级的**抢占式调度**。
- **系统开销极低**，通常采用微内核架构，上下文切换时间极短。
- 多采用**静态资源分配**，极少在运行时动态分配内存，以消除内存碎片和时间的不确定性。
- 具有极高的**系统可靠性**与故障隔离能力。

(2) 详细叙述：极强的确定性 (Determinism)

- **概念内涵：** 在RTOS中，确定性是指系统对特定外部事件（如硬件中断、高优先级任务就绪）的响应时间是**绝对精确、可预测且有严格上限的（即满足最坏执行时间 WCET）**。
- **与GPOS的对比差异：** 通用操作系统（如Windows/Linux）追求的是系统的整体吞吐量和平均性能，当系统负载过高或进行后台任务（如垃圾回收、页面置换）时，对用户的响应会出现不可预知的卡顿和延迟。而RTOS无论面对多大的系统压力，都必须在规定的微秒或纳秒级时间窗口内完成关键动作。
- **实现机制及意义：** RTOS通过禁用虚拟内存换页机制（直接在物理RAM中运行）、极度压缩内核“关中断”的代码执行时间等手段来实现这种确定性。在航空航天、汽车刹车系统（ABS）等工业控制场景中，错过控制周期（Deadline）的微小延迟都会导致灾难性后果，因此“确定性”是RTOS安身立命的根本。